



Face Recognition system

- Facial-scan technology utilizes **distinctive features** of the human face in order to verify or identify individuals.
- Facial-scan currently plays a role in the biometric marketplace in **1:N identification applications**
- **Applications:** ID card systems, in booking stations, and for various types of surveillance operations, facial-scan's most successful implementations take place in environments where cameras and imaging systems are already present.



Strength of Face Scan's

- It has the ability to leverage existing image acquisition equipment.
- It can search against static images such as driver's license photographs.
- It is the only biometric able to operate without user cooperation.



Weakness of Facial Scan's

- Changes in **acquisition environment** reduce matching accuracy.
- Changes in **physiological characteristics** reduce matching accuracy.
- It has the potential for privacy abuse due to noncooperative enrollment and identification capabilities.



Components of Facial Scan System

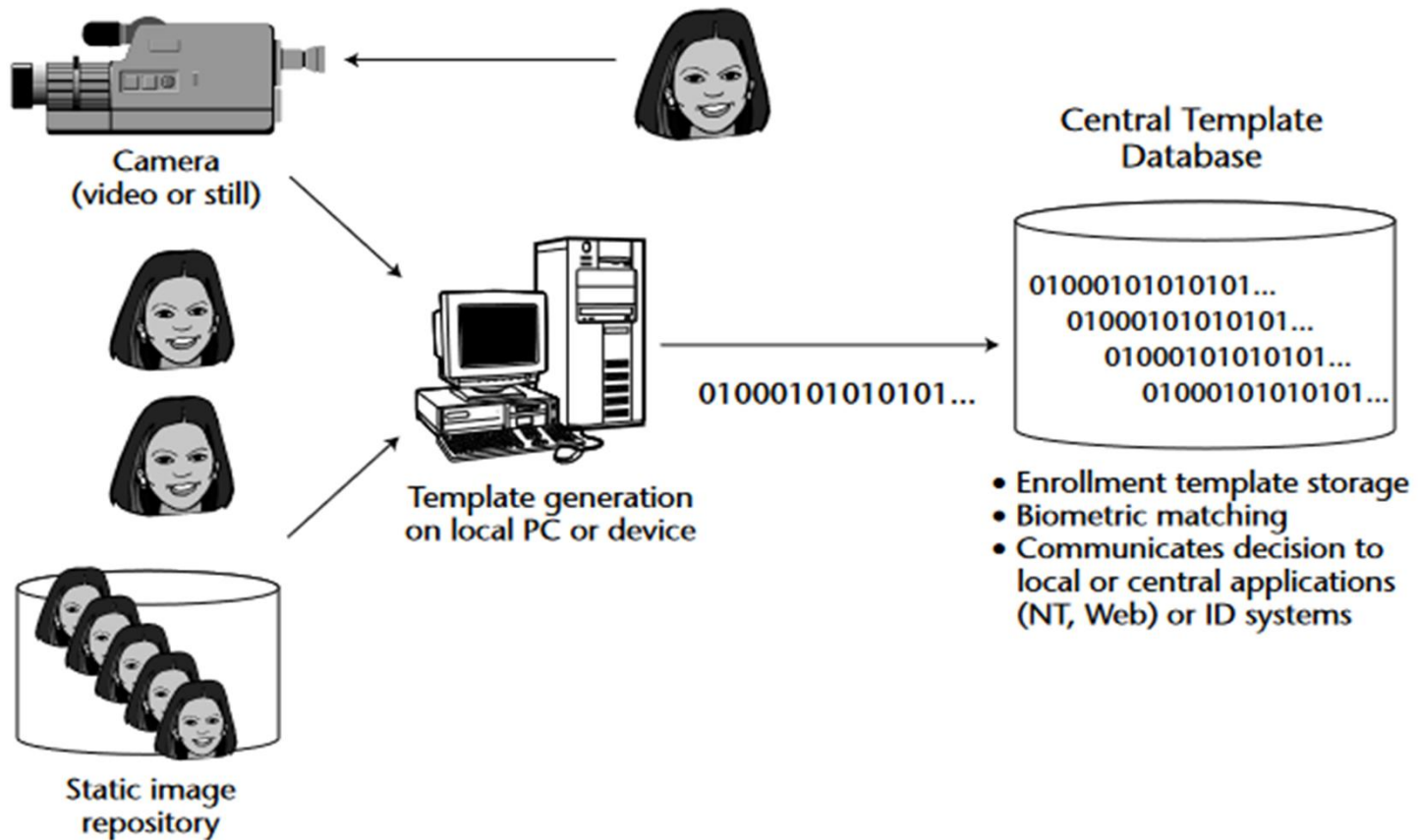


Figure 5.1 Components of a facial-scan system.



Components of Facial Scan System

- Facial-scan systems can range from software-only solutions that process images acquired through existing closed-circuit television (CCTV) cameras to full fledged acquisition and processing systems, including cameras, workstations, and back-end processors.
- In some facial-scan systems, the core technology is optimized to work with specific cameras and acquisition devices.



Components of Facial Scan System

- Basic Components

Face Location
Engine

- which finds and tracks faces in a field of view

Face
recognition engine

- which compares these faces—are normally located on the same PC or device, working in close conjunction

Facial-scan template matching normally takes place on a local or central PC, although the core technology has been embedded within devices such as personal digital assistants (PDAs) and mobile phones



How Facial-Scan Technology Works



Image acquisition: 2D vs. 3D

- **2D Imaging:** Relies on standard RGB cameras to capture flat, two-dimensional images. Systems extract measurements based on pixel contrast and visible light.
- **3D & Infrared:** illuminator bathes the face in invisible infrared light, and a dot projector casts over 30,000 infrared dots. An infrared camera reads these dots to generate a 3D depth map.
- Eg. Apple faceID



Image Acquisition

- Facial-scan technology can acquire faces from almost any **static camera or video system** that generates images of sufficient quality and resolution.
- **High-quality enrollment is essential** to eventual verification and identification; enrollment images define the facial characteristics to be used in all future authentication events.
- **Distance from camera reduces facial size** and therefore image resolution, though the technology is capable of magnifying smaller faces to increase the possibility of template generation.



Image Acquisition

- The problem of **lighting** is more severe.
- facial- scan technologies are generally **unable to acquire images** that are somewhat **overexposed or underexposed**.
- Systems with **automatic gain control**, able to adjust for different lighting conditions and skin tone, are more likely to present images from which facial-scan solutions can acquire faces.
- Select Hispanic, black, and Asian individuals can be more difficult to enroll and verify in some facial-scan systems because acquisition devices are not always optimized to acquire darker faces.



Image Processing

Image Cropping

- Images are cropped such that the ovoid facial image remains

Colour Conversion

- color images are normally converted to black and white in order to facilitate initial comparisons based on grayscale characteristics.

Normalization

- Facial images are then normalized to overcome variations in orientation and distance.
- The process of normalization is related to the face alignment stage, which aims to find face landmarks (point markers of facial features).



Distinctive characteristic location

If you look in the mirror, you can see that your face has certain distinguishable landmarks.

- These are the **peaks and valleys** that make up the different facial features.
- There are about **80 nodal points** on a human face.

Here are a few of the nodal points that are measured by the software:

- Distance between eyes
- Width of nose
- Depth of eye sockets
- Cheekbones
- Jaw line
- Chi

These nodal points are measured to create a **numerical code, a string of numbers, that represents the face in a database.** This code is called a **faceprint or embeddings**

Faceprint:

- We know that Facial features are converted into a dense numerical vector—a faceprint
- e.g., a 128-dimensional floating-point array. This mathematically maps the geometric properties and distinct features of the face into a unique sequence

Face Detection & Alignment

- **Detection:** Algorithms (such as Haar cascades or deep learning-based Convolutional Neural Networks, or CNNs) scan the image to isolate the face from the background by evaluating object textures and contrast.
- **Alignment:** To ensure consistent feature extraction regardless of head tilt, the system normalizes the image by adjusting the detected face so the eyes, nose, and lips are aligned on a standard axis.



Face Recognition Techniques

- Face recognition algorithms can be classified into **two broad categories** according to **feature extraction schemes** for face representation:

**Face
Recognition
Techniques**

- **Feature based methods**

- Properties and geometric relations such as the areas, distances, and angles between the facial feature points are used as descriptors for face recognition.

**Face
Recognition
Techniques**

- **Appearance based methods**

- Appearance-based methods consider the global properties of the face image intensity pattern



Face Recognition Techniques

- Typically appearance-based face recognition algorithms proceed by computing basis vectors to represent the face data efficiently.
- In the next step, the faces are projected onto these vectors and the projection coefficients can be used for representing the face images.
- Popular algorithms such as PCA, LDA, ICA, LFA, Correlation Filters, Manifolds and Tensor faces.

- **Linear discriminant analysis (LDA).** LDA aims to create new components that separate clusters based on a categorical feature.
- **Independent component analysis (ICA).** ICA aims to retrieve original features that are mixed together in a linear combination in the input dataset.
- **Local Feature Analysis (LFA)** LFA uses the person's facial contour lines calculated through the detection process to examine and classify line-segment types.
- **Principal Component Analysis (PCA)**

- Principal component analysis, or PCA, is a dimensionality reduction method that is often used to reduce the dimensionality of large data sets, by transforming a large set of variables into a smaller one that still contains most of the information in the large set.
- **reduce the number of variables of a data set, while preserving as much information as possible.**

Matching & Verification

- The system evaluates similarity by calculating the distance (e.g., Euclidean distance or Cosine similarity) between the captured face's vector and the stored template's vector

$$\text{Distance} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

If the calculated distance falls below a predetermined similarity threshold (indicating a high probability of a match), the identity is successfully authenticated

Liveness Detection and Security

- To prevent spoofing with photos or masks, systems use Presentation Attack Detection (PAD).
- They verify liveness by tracking micro-movements, eye blinks, or analyzing depth variations.
- Biometric templates are also secured using **FIDO2/WebAuthn** decentralized architectures, where data stays locally encrypted in a secure hardware enclave.
- **FIDO (Fast Identity Online)** is managed by the FIDO Alliance, a non-profit industry consortium founded in 2012 by companies including PayPal and Lenovo.

FIDO Key Architectural Components

- Relying Party (RP): The application server that requests authentication and validates cryptographic signatures.
- WebAuthn API: A standard JavaScript API built into modern browsers and operating systems. It allows the client application to talk to the authenticator.
- Authenticator: The hardware or software entity that generates and stores cryptographic keys. This can be an Internal Authenticator (e.g., Apple Secure Enclave, Android Keystore) or an External/Cross-Platform Authenticator (e.g., YubiKey via USB/NFC).
- CTAP2 (Client-to-Authenticator Protocol): The protocol that allows an external hardware key or smartphone to communicate with a browser or OS client over USB or Bluetooth

Use Case 1: Mobile App Login (Bank)

- **Initial Setup:** The user logs in with biometric/traditional login
- **Key Creation:** The app invokes the native OS API (Android BiometricPrompt or iOS LocalAuthentication). The device creates a FIDO2 key pair tied specifically to that banking app.
- **Daily Login:**
 1. The user opens the app. The app requests a login challenge from the bank server.
 2. The phone prompts the user for a face scan.
 3. The native hardware validates the face locally. It never sends your biometric image or data to the bank server.
 4. Upon successful local match, the Secure Enclave releases the private key to sign the server's challenge.
 5. The signature is sent to the bank server, which validates it and instantly logs the user in.

Security & Operational Advantages

- **Phishing-Proof:** The key pair is bound to the specific app ID and domain. A fake phishing app cannot intercept or reuse the signature.
- **Zero Trust Biometrics:** The bank server never holds, sees, or processes the user's facial data. It only receives a cryptographic verification signature

Use Case 2: College Attendance System

- Implementing face biometrics for college classroom attendance requires a centralized, one-to-many (1:N) processing model. This structure differs significantly from the localized 1:1 FIDO2 model



Principal Component Analysis (PCA)

- Principal Component Analysis (PCA) is an **unsupervised, non-parametric statistical technique** primarily used for **dimensionality reduction in machine learning**.
- PCA is a way of reducing the dimensions of a **large dataset by transforming it into a smaller dataset**, but ensuring that the smaller dataset **contains more information than the larger dataset**.
- By reducing the dataset, we are also **reducing the accuracy**. However, PCA works on the **principle of trading little accuracy for simplicity**.
- This is because **smaller datasets are easier to explore and visualize**, thus making data analysis easier and faster for machine learning algorithms.

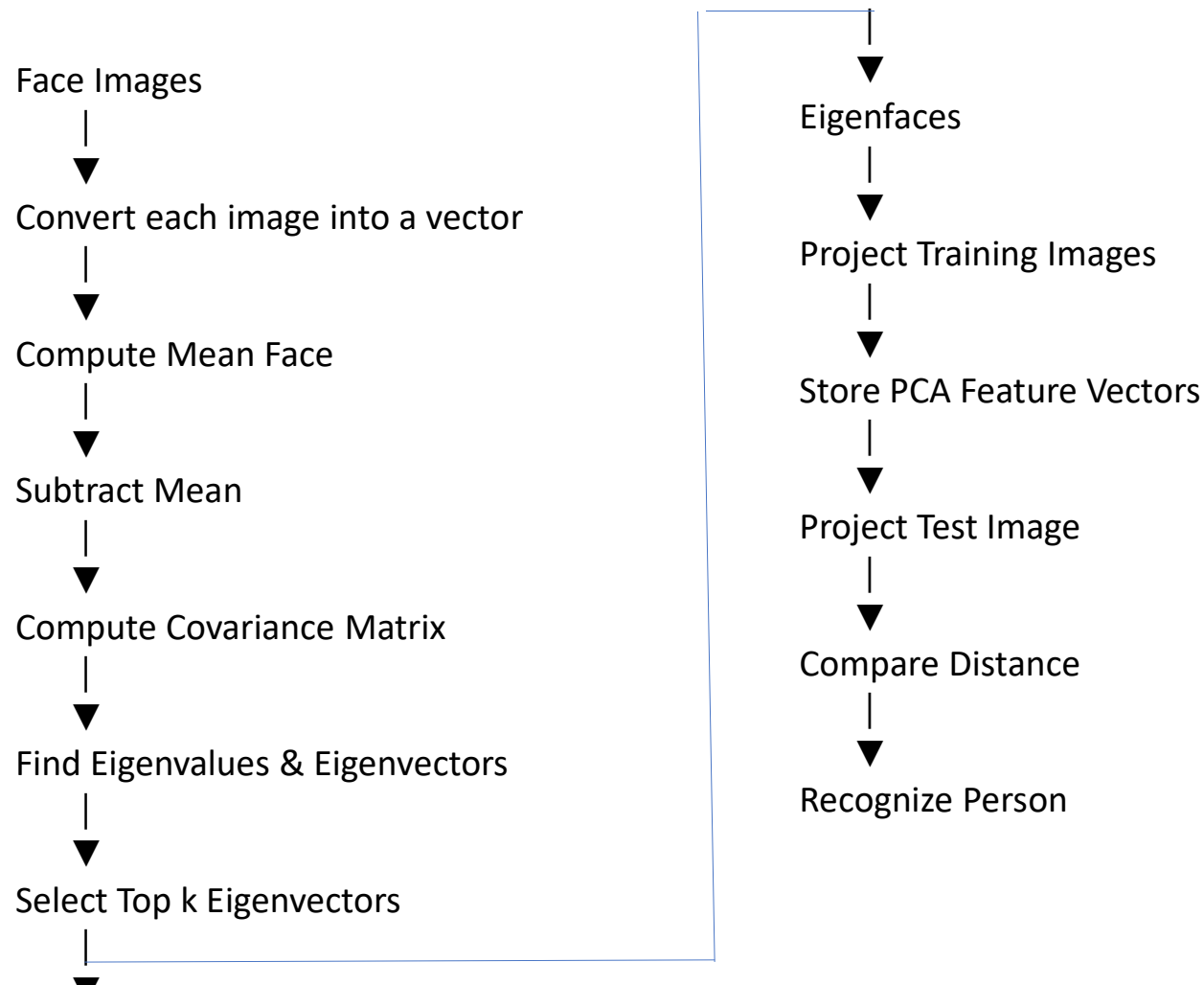
Recall : PCA (We studied in AI&NN)

- **Principal Component Analysis (PCA)** is one of the earliest and most widely used dimensionality reduction techniques in **facial biometric systems**.
- It is popularly known as the **Eigenfaces** method, proposed by Turk and Pentland (1991).
- The main idea is that instead of storing every pixel of a face image, PCA extracts only the most important information (principal components), thereby reducing computation while preserving facial characteristics.

Why PCA is Required in Facial Biometrics

- Consider a grayscale face image of size **100 × 100 pixels**.
- Total pixels = ? $100 \times 100 = \mathbf{10,000}$ features
- Every image becomes a vector of length **10,000**
- Suppose we have 500 face images
- Each image has 10,000 pixel values
- The dataset size becomes $500 \times 10,000$.
- Training directly on these 10,000-dimensional vectors is computationally expensive , memory intensive , highly redundant because neighboring pixels are strongly correlated.
- PCA solves this problem.

Overall PCA Workflow in Face Recognition



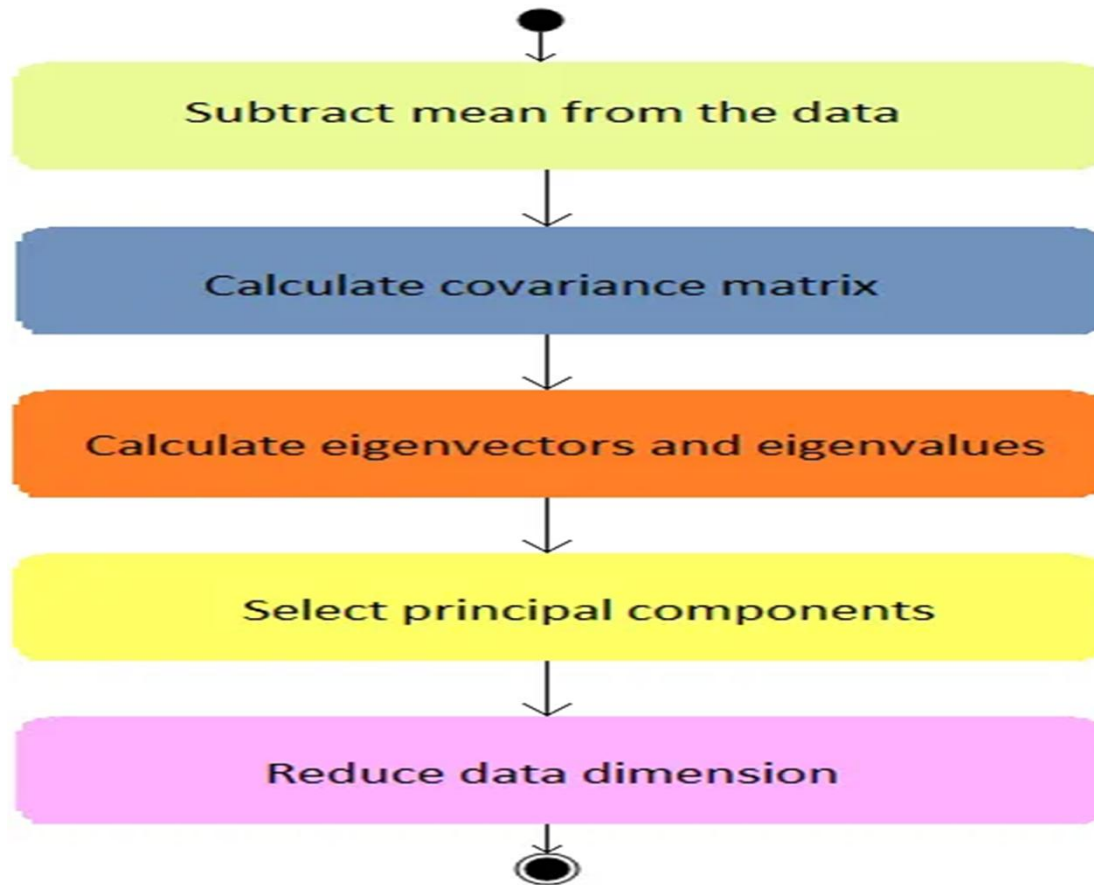


Principal Component Analysis (PCA)

- **Eigenvectors and eigenvalues** are the linear algebra concepts that are used to **compute the covariance matrix** to determine the principal component of the data.
- Why PCA is popular?
- The reason why it chooses the direction of bigger variance because it could significantly compress the dimension without losing a lot of information.



Steps In PCA



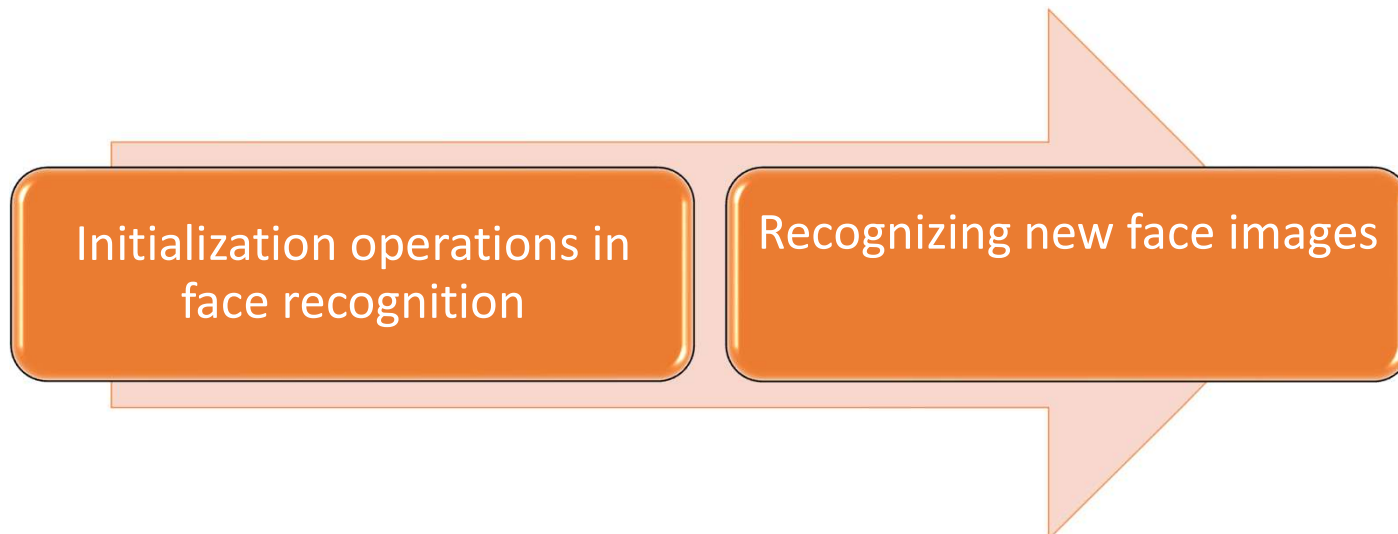


How PCA is used in Face recognition system ?

- **How PCA is used in Face recognition system ?**

There are 2 Steps of Eigenface (PCA) Approach: Initialization operations in face recognition, Recognizing new face images

-





How PCA is used in Face recognition system ?

Steps

► Initialization Operations in Face Recognition

- ❖ Prepare the Training Set to Face Vector
- ❖ Normalize the Face Vectors
- ❖ Calculate the Eigen Vectors
- ❖ Reduce Dimensionality
- ❖ Back to original dimensionality
- ❖ Represent Each Face Image a Linear Combination of all K Eigenvectors

► Recognizing An Unknown Face

Assume we have 4 training images.

- Each image is only **2 × 2 pixels** (for simplicity).

Image 1	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$	Image 3	$\begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix}$
Image 2	$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$	Image 4	$\begin{bmatrix} 4 & 5 \\ 6 & 7 \end{bmatrix}$

1,2,3,4 ... 7 represent the pixel intensity values of an image.

In a real face image, every pixel stores a brightness (grayscale) value.

0 → Black

255 → White

128 → Gray

For simplicity, instead of using values from 0–255, I used small numbers (1–7) so that the PCA calculations are easy to understand.

Step 1 Convert Images into Vectors

Flatten row-wise.

Image 1

[1, 2, 3, 4]

Image 2

[2, 3, 4, 5]

Image 3

[3, 4, 5, 6]

Image 4

[4, 5, 6, 7]

Training matrix

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 3 & 4 & 5 & 6 \\ 4 & 5 & 6 & 7 \end{bmatrix}$$

Each row represents one face.

Step 2 Compute Mean Face (Mean of each column)

Column 1	$(1 + 2 + 3 + 4)/4 = 2.5$
Column 2	3.5
Column 3	4.5
Column 4	5.5
Mean face	$\mu = [2.5, 3.5, 4.5, 5.5]$

Mean face is

Mean

$$\Psi = \begin{bmatrix} 2.5 \\ 3.5 \\ 4.5 \\ 5.5 \end{bmatrix}$$

Step 3: Normalize
(Subtract the mean)

$$\Phi_1 = \begin{bmatrix} -1.5 \\ -1.5 \\ -1.5 \\ -1.5 \end{bmatrix}$$

$$\Phi_2 = \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{bmatrix}$$

$$\Phi_3 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

$$\Phi_4 = \begin{bmatrix} 1.5 \\ 1.5 \\ 1.5 \\ 1.5 \end{bmatrix}$$

Form Matrix A

Arrange normalized vectors as columns.

$$A = \begin{bmatrix} -1.5 & -0.5 & 0.5 & 1.5 \\ -1.5 & -0.5 & 0.5 & 1.5 \\ -1.5 & -0.5 & 0.5 & 1.5 \\ -1.5 & -0.5 & 0.5 & 1.5 \end{bmatrix}$$

Step 4 Covariance Matrix

Covariance measures how features vary together.

PCA computes

$$C = \frac{1}{n-1} X^T X$$

Instead of looking at individual pixels, covariance reveals relationships between them.

Step 5 Eigenvalues and Eigenvectors

- Suppose the covariance matrix gives
- Eigenvalues
- [18, 2, 0.4, 0.1]
- Largest eigenvalue 18, Second is 2
- Since the first two eigenvalues explain almost all the variance,
- we keep only Eigenface 1 , Eigenface 2
- Discard Eigenface 3 and Eigenface 4

What is an Eigenface?

- An eigenface is **not a real face**.
- It is a mathematical pattern representing a major direction of variation in facial appearance.

Suppose

Eigenface 1

$$v_1 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

Suppose the second eigenvector is

$$v_2 = \begin{bmatrix} 0.5 \\ -0.5 \\ 0.5 \\ -0.5 \end{bmatrix}$$

These two vectors form our **Eigenface Space**.

Step 6: Compute the Weights of Every Training Face

- Formula

$$w = v^T \Phi$$

Each face gets one weight for each Eigenface.

Face 1

Normalized

$$(-1.5, -1.5, -1.5, -1.5)$$

Weight on Eigenface 1

$$\begin{aligned}w_1 &= 0.5(-1.5) + 0.5(-1.5) + 0.5(-1.5) + 0.5(-1.5) \\&= -3\end{aligned}$$

Weight on Eigenface 2

$$\begin{aligned}0.5(-1.5) + (-0.5)(-1.5) + 0.5(-1.5) + (-0.5)(-1.5) \\= -0.75 + 0.75 - 0.75 + 0.75 = 0\end{aligned}$$

Therefore

Face 1

$$(-3, 0)$$

Face 2

Normalized

$$(-0.5, -0.5, -0.5, -0.5)$$

Weight 1

$$0.5(-0.5) \times 4 = -1$$

Weight 2

$$0$$

Face 2

$$(-1, 0)$$

Face 3

Normalized

$$(0.5, 0.5, 0.5, 0.5)$$

Weight 1

$$1$$

Weight 2

$$0$$

Face 3

$$(1, 0)$$

Face 4

Normalized

$$(1.5, 1.5, 1.5, 1.5)$$

Weight 1

$$3$$

Weight 2

$$0$$

Face 4

$$(3, 0)$$

Final Training Database

Person	Weight Vector
Face 1	$(-3,0)$
Face 2	$(-1,0)$
Face 3	$(1,0)$
Face 4	$(3,0)$

Testing : Recognition Phase

Now suppose a new image arrives.

New face

$$\begin{bmatrix} 2.2 & 3.2 \\ 4.2 & 5.2 \end{bmatrix}$$

Convert to vector

$$\Gamma = \begin{bmatrix} 2.2 \\ 3.2 \\ 4.2 \\ 5.2 \end{bmatrix}$$

Testing : Recognition Phase

Normalize

Subtract mean

$$\Phi = \begin{bmatrix} -0.3 \\ -0.3 \\ -0.3 \\ -0.3 \end{bmatrix}$$

Compute Weights

Eigenface 1

$$w_1 = 0.5(-0.3) \times 4 = -0.6$$

Eigenface 2

$$0$$

Weight vector

$$(-0.6, 0)$$

Euclidean Distance

Distance to Face 1

$$\sqrt{(-0.6 + 3)^2} = 2.4$$

Distance to Face 2

$$\sqrt{(-0.6 + 1)^2} = 0.4$$

Distance to Face 3

$$\sqrt{(-0.6 - 1)^2} = 1.6$$

Distance to Face 4

$$\sqrt{(-0.6 - 3)^2} = 3.6$$

Decision

Smallest distance 0.4.
Therefore

- **New face is recognized as Face 2.**